



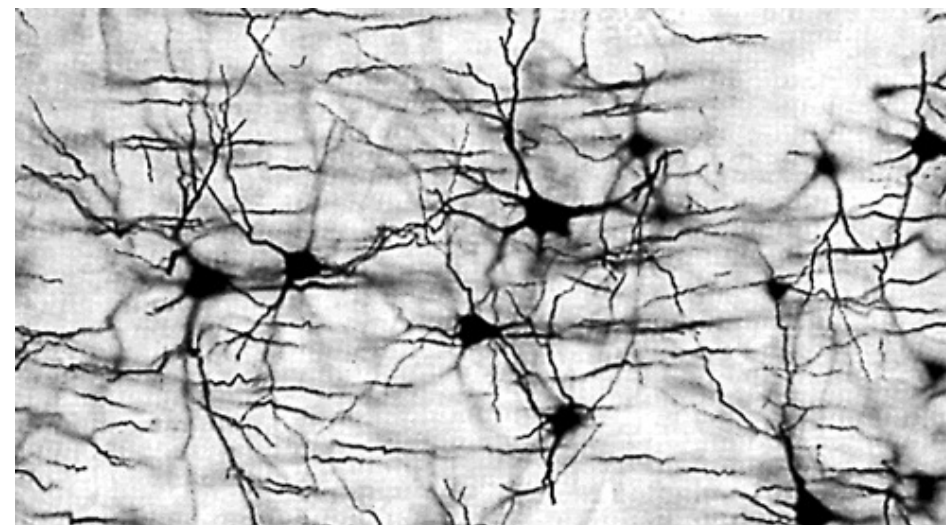
02456 – Week 6

Transformers

Jes Frellsen

Technical University of Denmark

06 October 2025



Menu of the day

- Week 1: Neural nets
- Week 2: Learning
- Week 3: Tricks of The Trade
- Week 4: CNNs
- Week 5: RNNs
- Week 6: Transformers
- **Week 7: Unsupervised**
- Week 8: Mini-project

Architectures for
sequential data

Lecture

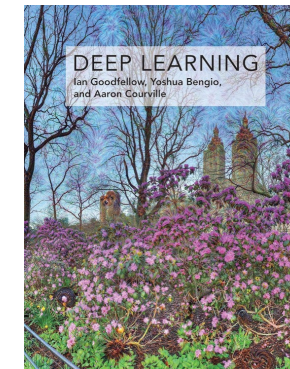
- Attention
- Transformers

Reading

- Chapter 12

Exercises

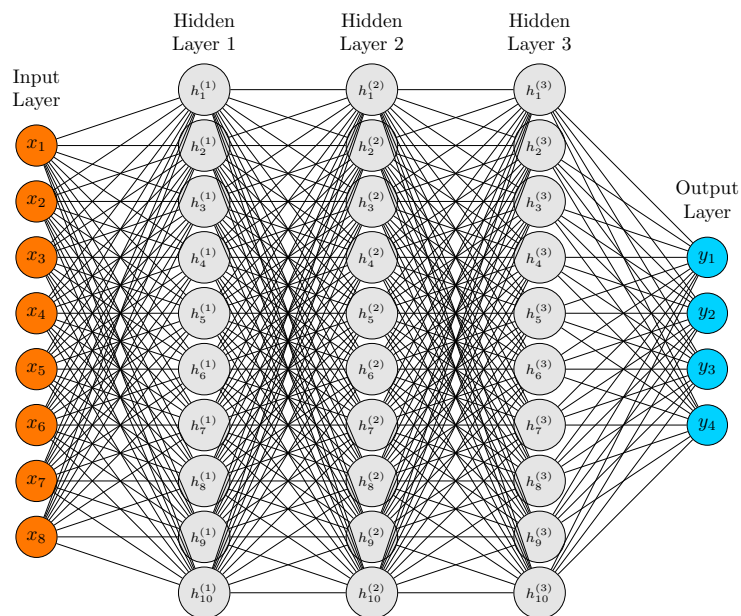
- **Notebooks 6.1:**
 - Language modelling
 - RNNs
 - Attention and Transformers
- **Problems:** 12.3, 12.10



Announcements

- **Project list** will be published Wednesday this week
 - **Project selection deadline:** Monday, Week 8, 23:59
- An **example of the multiple-choice exam** will be published during the project period

Recap: Deep learning



We can write the **output of layer ℓ** as

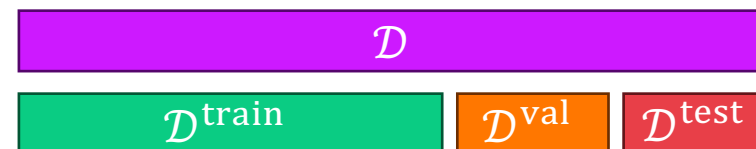
$$f^{(\ell)}(\mathbf{h}) = \sigma(W^{(\ell)}\mathbf{h} + \mathbf{b}^{(\ell)}),$$

where σ is a non-linear **activation function**.

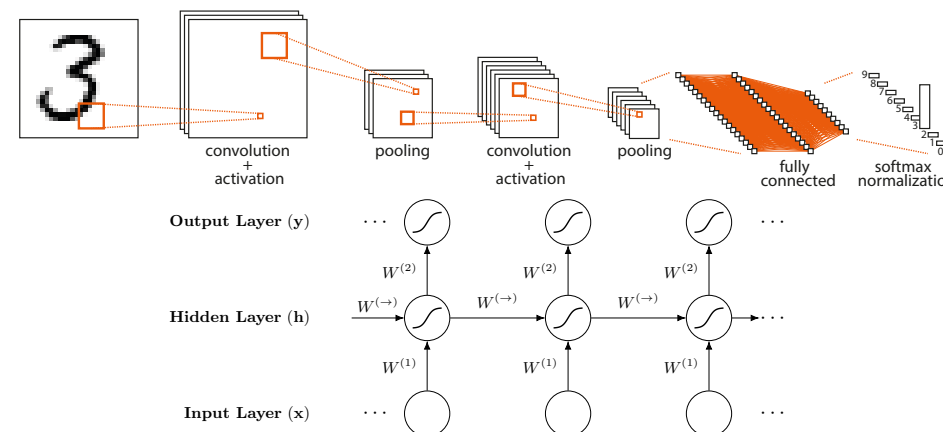
The **joint function** is then

$$f_{\phi}(\mathbf{x}) = \mathbf{y} = f^{(4)}\left(f^{(3)}\left(f^{(2)}\left(f^{(1)}(\mathbf{x})\right)\right)\right)$$

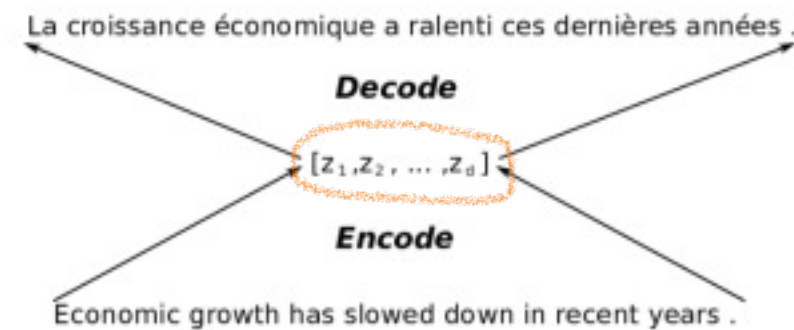
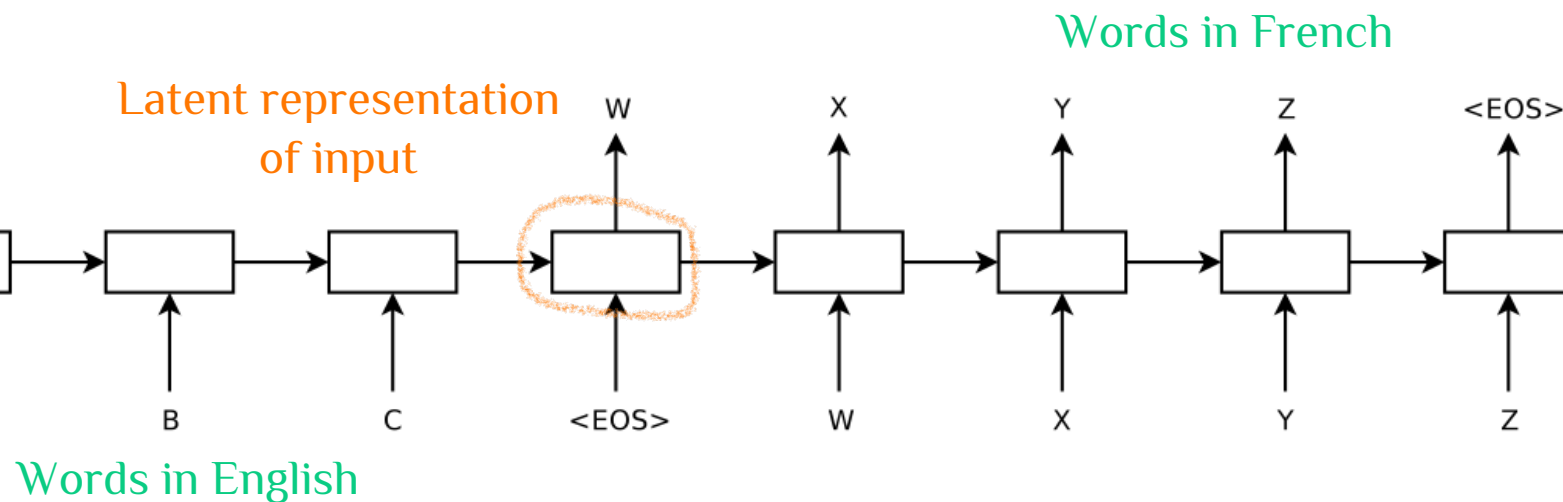
- A **loss function** $L(\phi)$ captures the **mismatch** of $f_{\phi}(\mathbf{x}_i)$ and $\mathbf{y}_i$: $\hat{\phi} = \operatorname{argmin}_{\phi} L(\phi)$
- Use the **NLL** $L(\phi) = -\sum_i \log p(\mathbf{y}_i | f_{\phi}(\mathbf{x}_i))$
- Use **gradient descent** to find $\hat{\phi}$
- Split **data set** in three



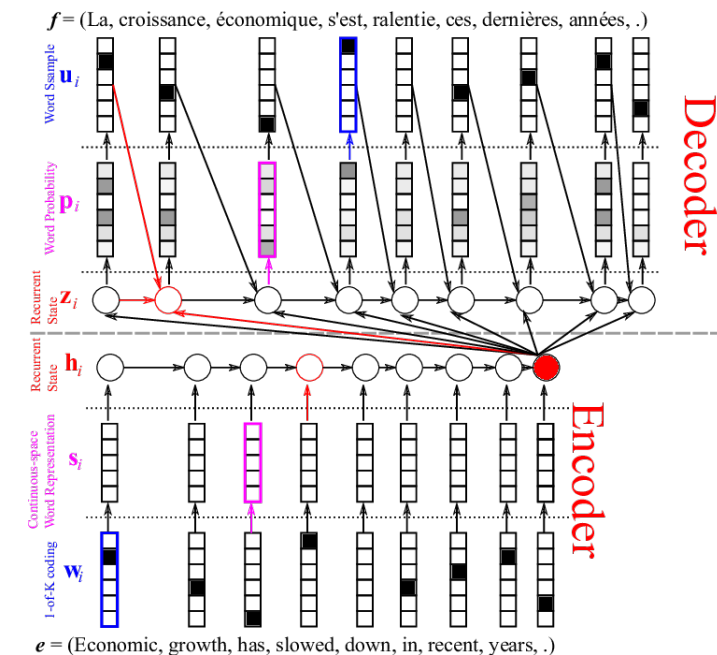
- Special architectures for images and sequences



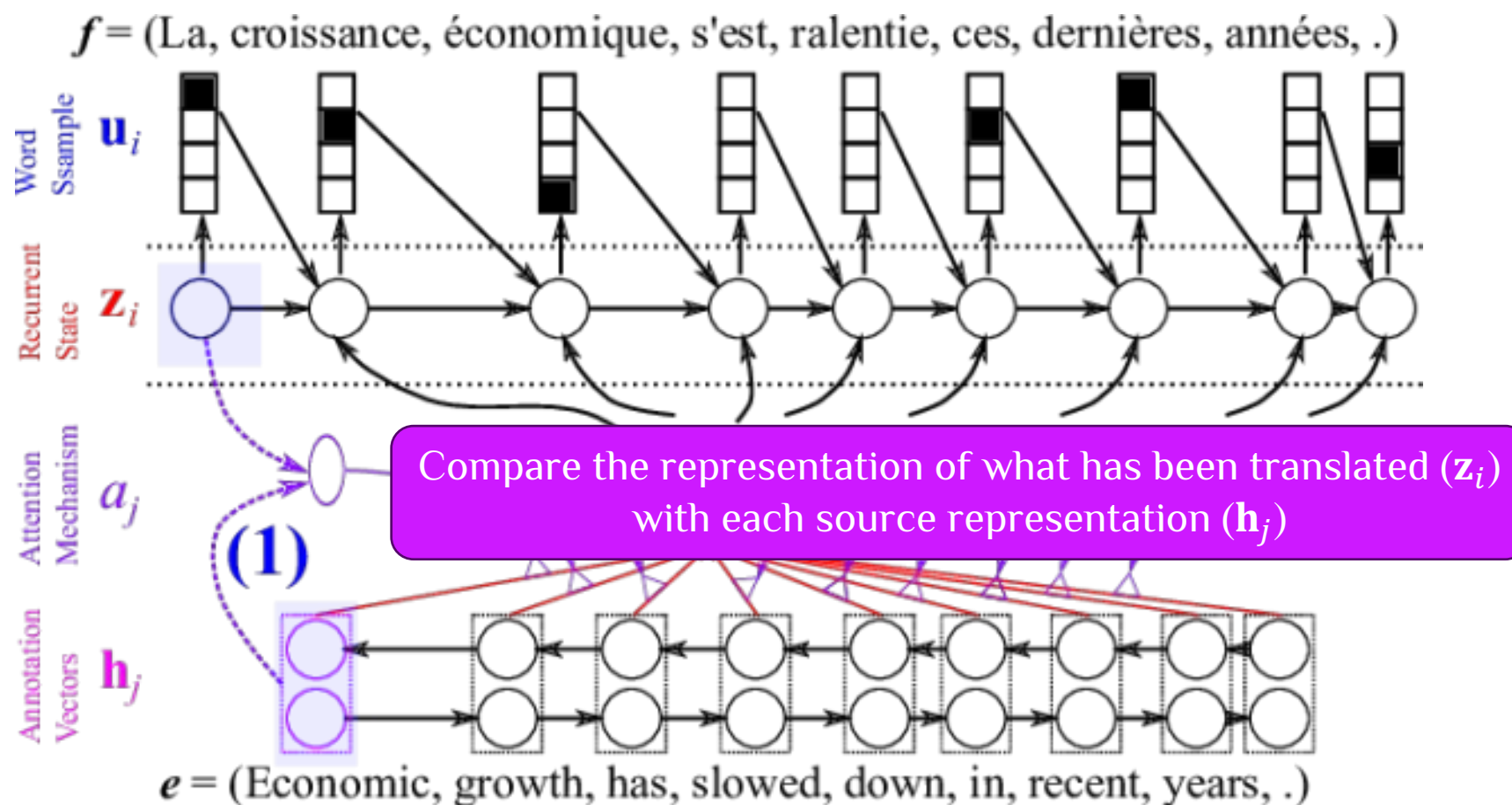
RNN Encoder-decoder for machine translation



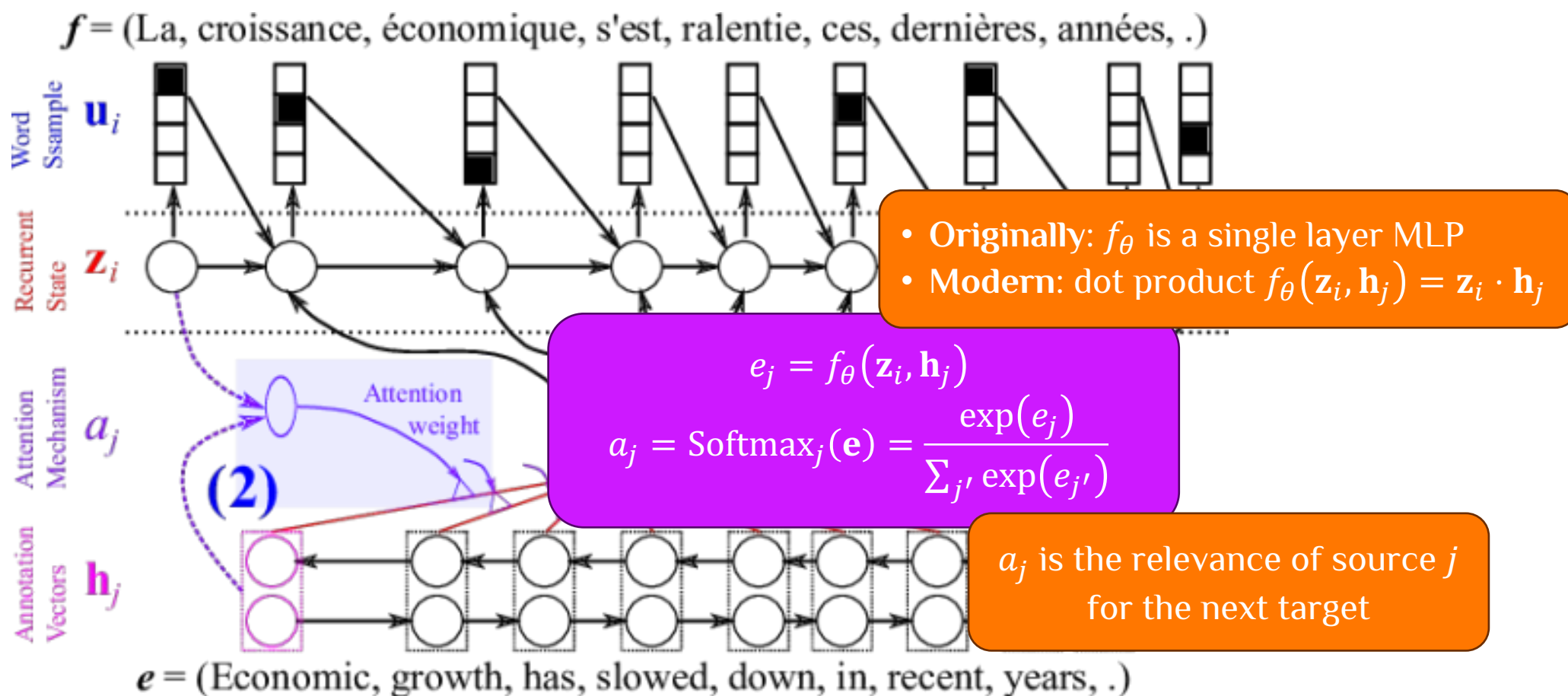
- Issue: **latent** (context) vector must encode **all source details**



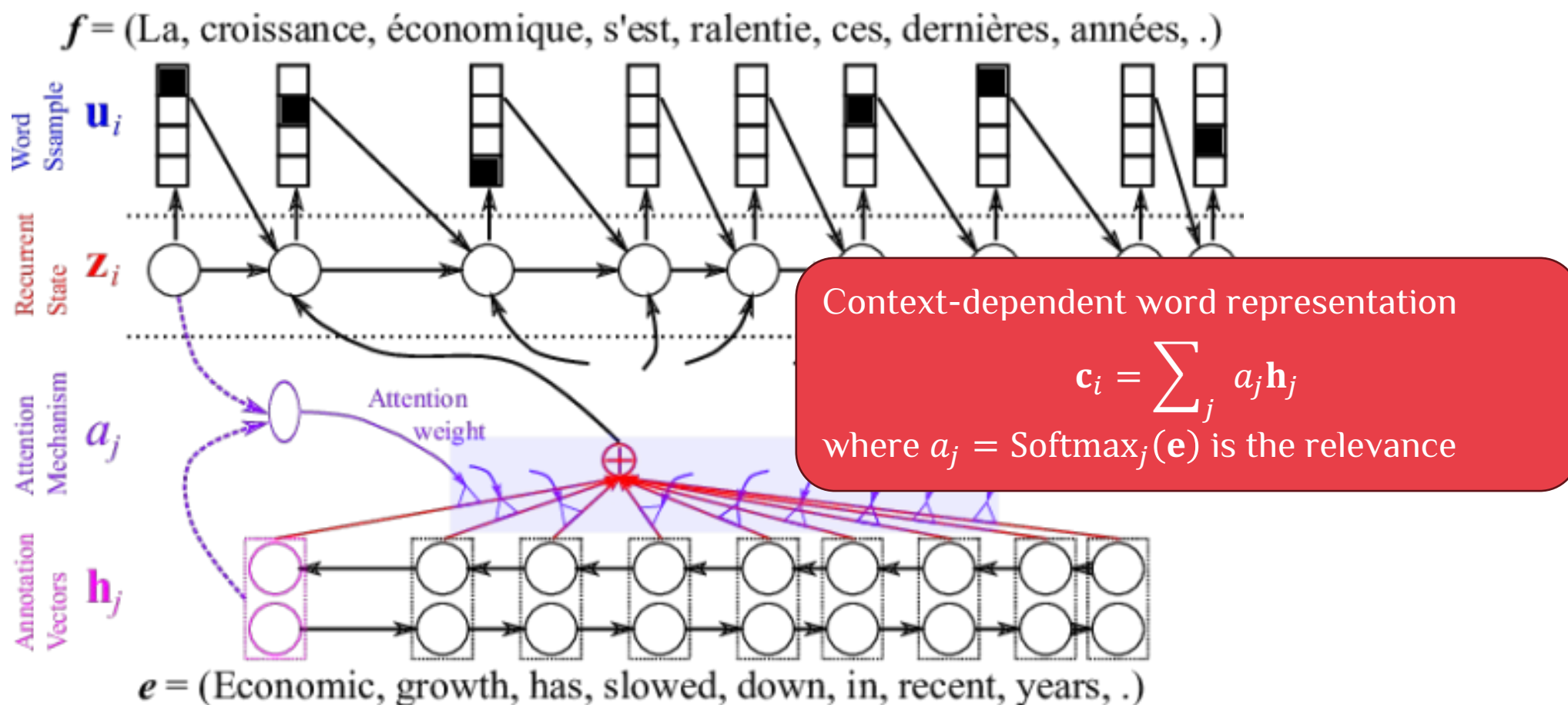
Adding attention to RNNs



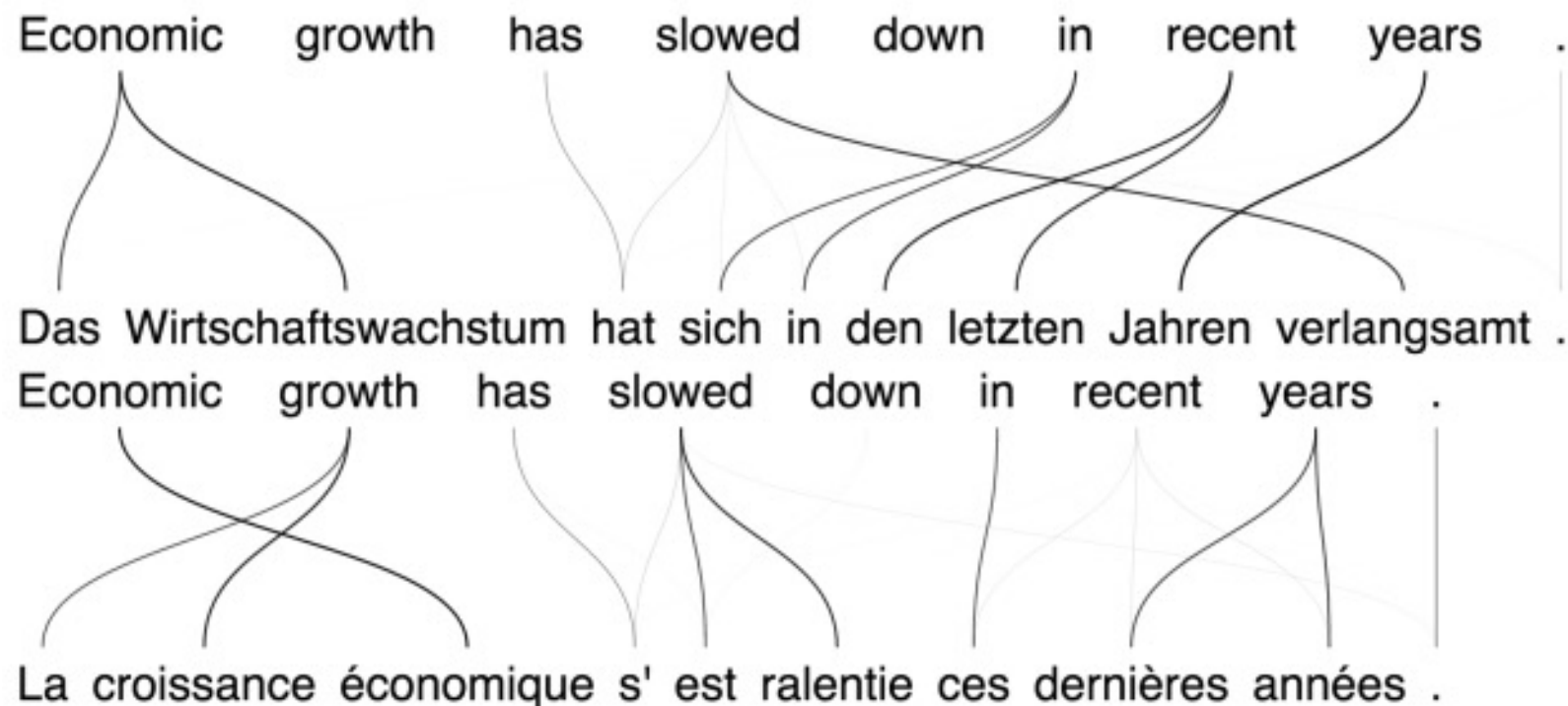
Adding attention to RNNs



Adding attention to RNNs



RNN attention for machine translation



RNN with attention over images for neural image caption



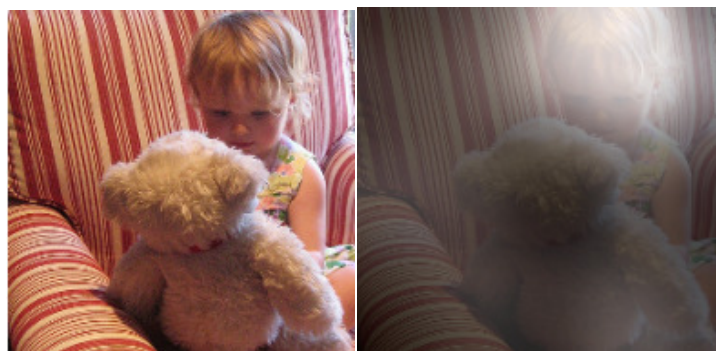
A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



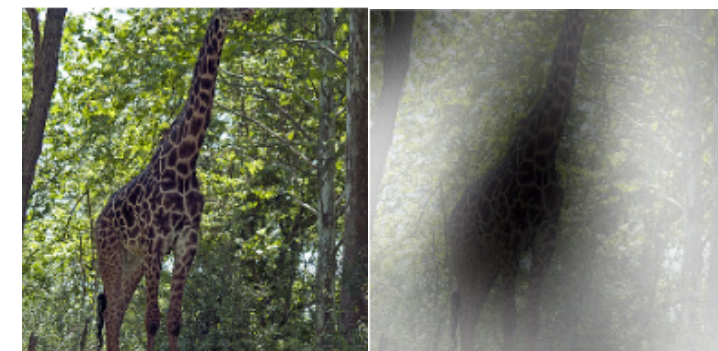
A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.



A group of people sitting on a boat in the water.



A giraffe standing in a forest with trees in the background.

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Turns out: we do not need the RNN

Attention and FNNs are enough

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer,



Transformer layers

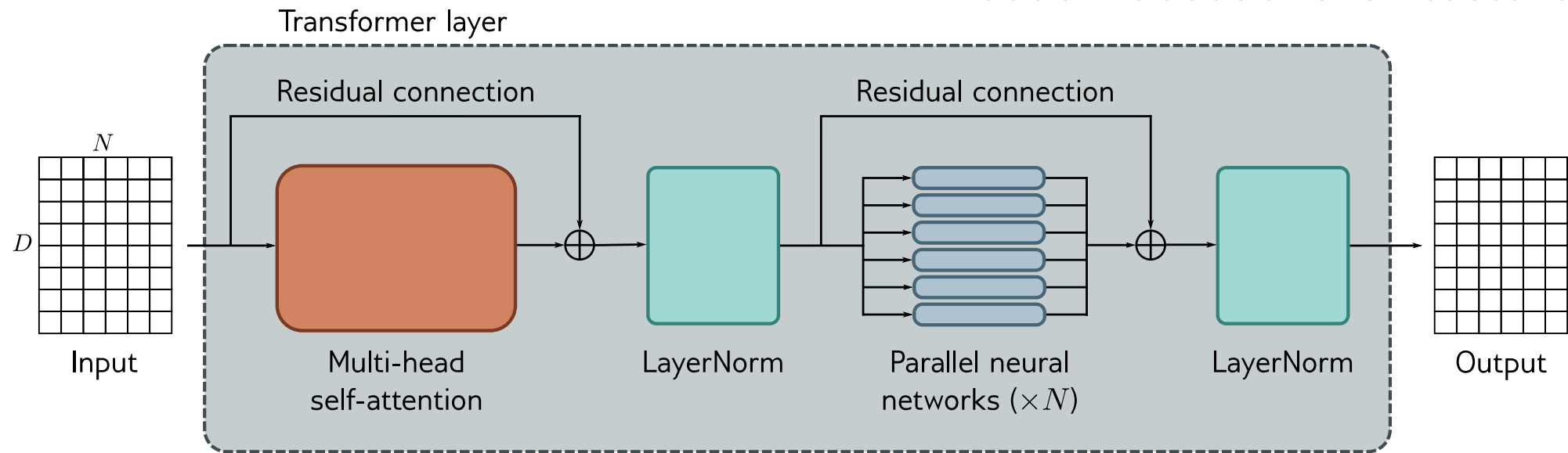
A **Transformer layer** consists of

- Multi-head self-attention
- Layer normalization
- Position-wise FFN
- Residual connections

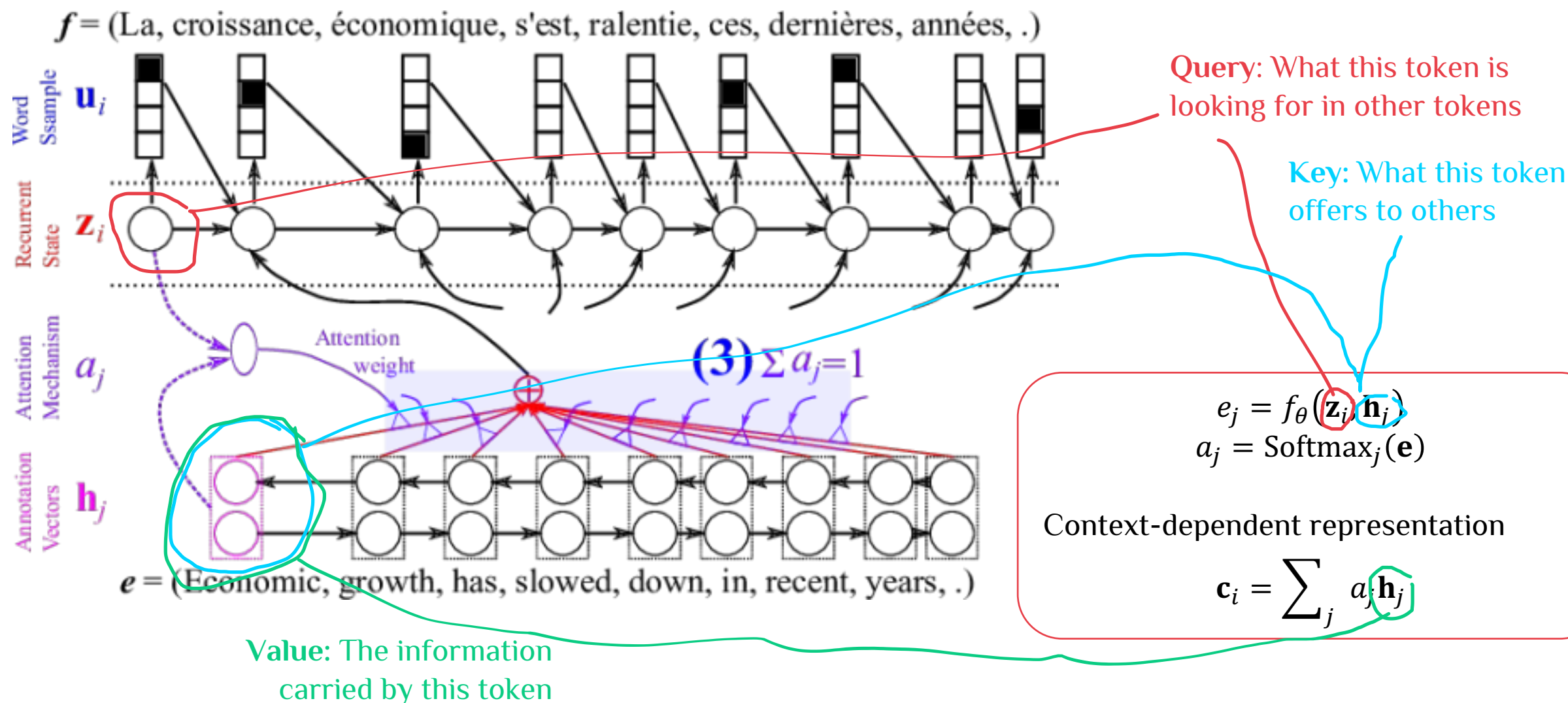
Stacked to build deep models

Used in

- Encoder-only models
- Decoder-only models
- Encoder–decoder architectures



Towards a generic attention layer



Towards dot-product attention layer

In an attention layer we input:

- A **query** vector $\mathbf{q} \in \mathbb{R}^{1 \times D}$
- A **key** sequence $K \in \mathbb{R}^{T' \times D}$
- A **value** sequence $V \in \mathbb{R}^{T' \times D'}$

The **output** $y \in \mathbb{R}^{D'}$ is a weighing of V_t

Attention:

$$e_{t'} = \mathbf{q} \cdot \mathbf{K}_{t'} \text{ or } \mathbf{e} = \mathbf{q} K^T$$
$$\mathbf{a} = \text{softmax}(\mathbf{q} K^T)$$

Weight sum of values

$$y = \sum_{t'} a_{t'} V_{t'} = \mathbf{a} V$$

Scaled dot-product attention

In an attention layer we input:

- A **query** sequence $Q \in \mathbb{R}^{T \times D}$
- A **key** sequence $K \in \mathbb{R}^{T' \times D}$
- A **value** sequence $V \in \mathbb{R}^{T' \times D'}$

The **output** $y \in \mathbb{R}^{D'}$ is a weighing of V_t

Attention:

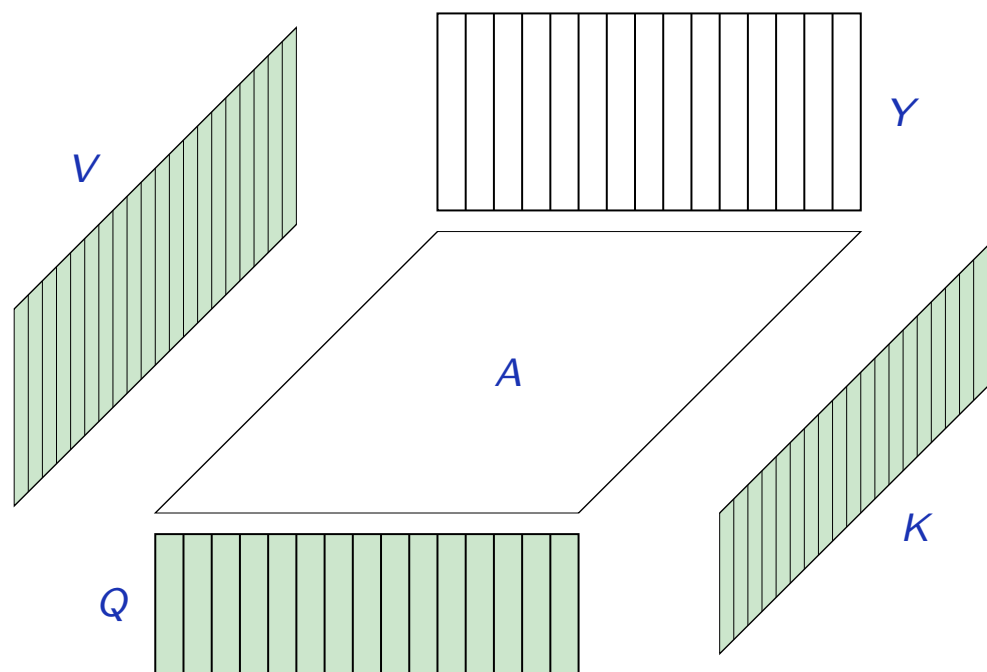
$$A = \text{softmax}_{\text{row}} \left(\frac{QK^T}{\sqrt{D'}} \right)$$

Weight sum of values:

$$Y = AV$$

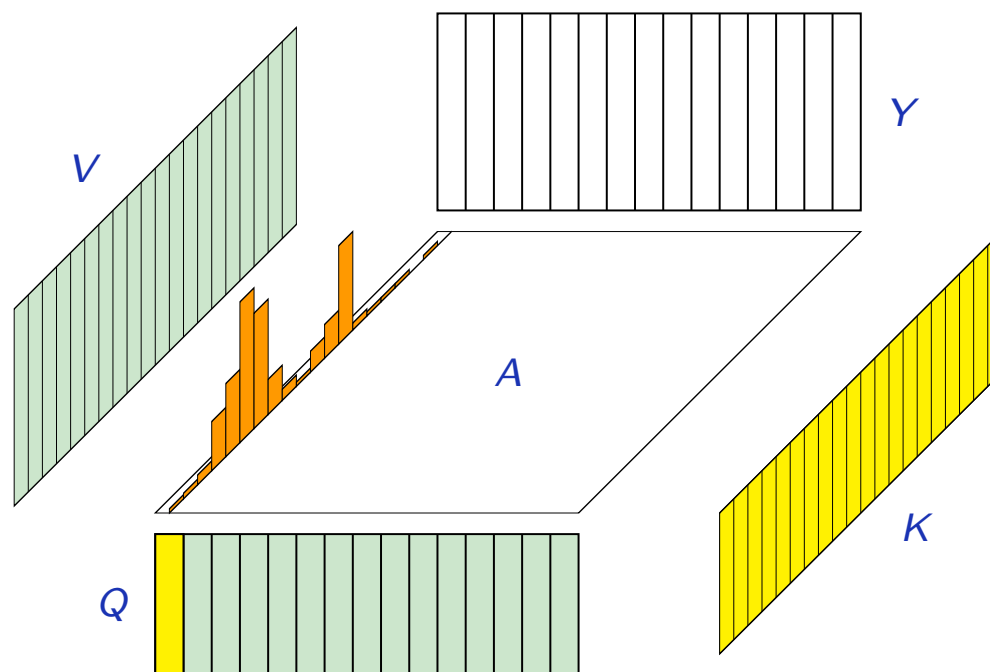
Scaled to avoid that largest value completely dominates

Scaled dot-product attention



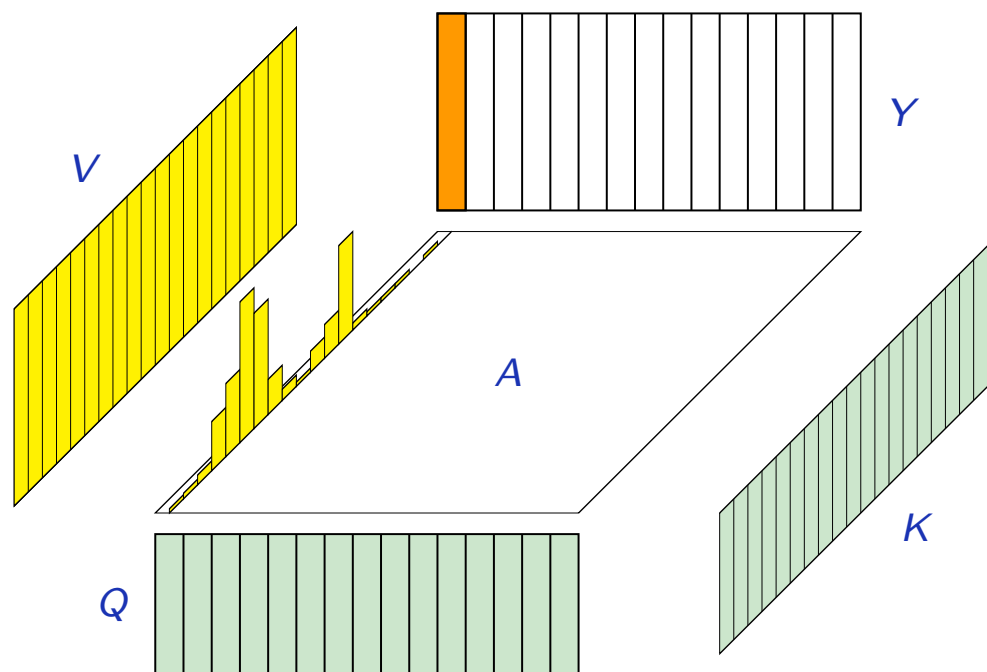
Scaled dot-product attention

$$A_i = \text{softmax} \left(\frac{KQ_i}{\sqrt{D}} \right)$$



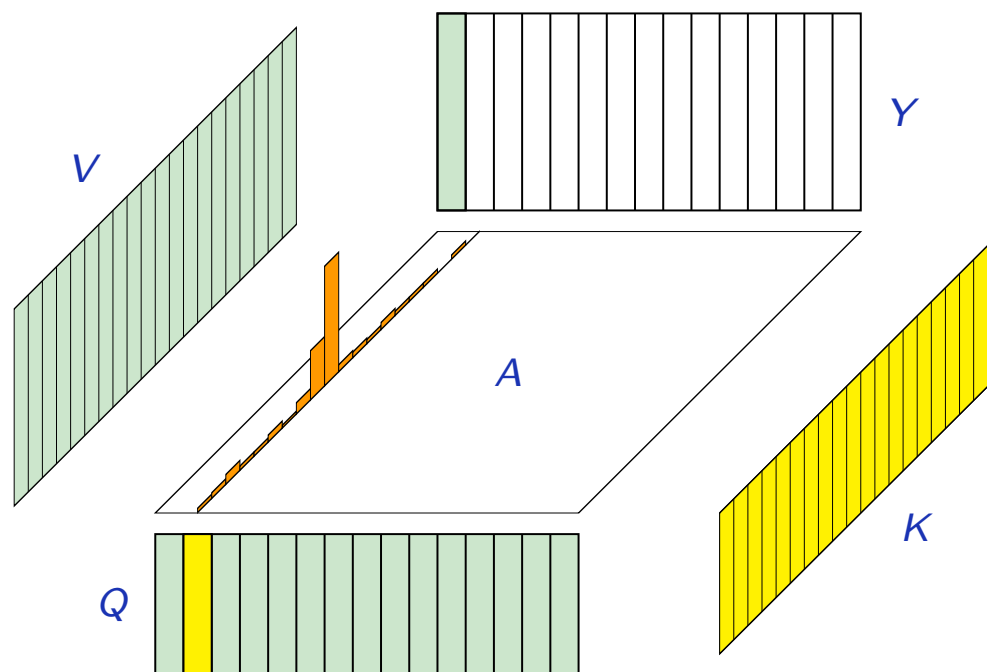
Scaled dot-product attention

$$Y_i = V^T A_i$$



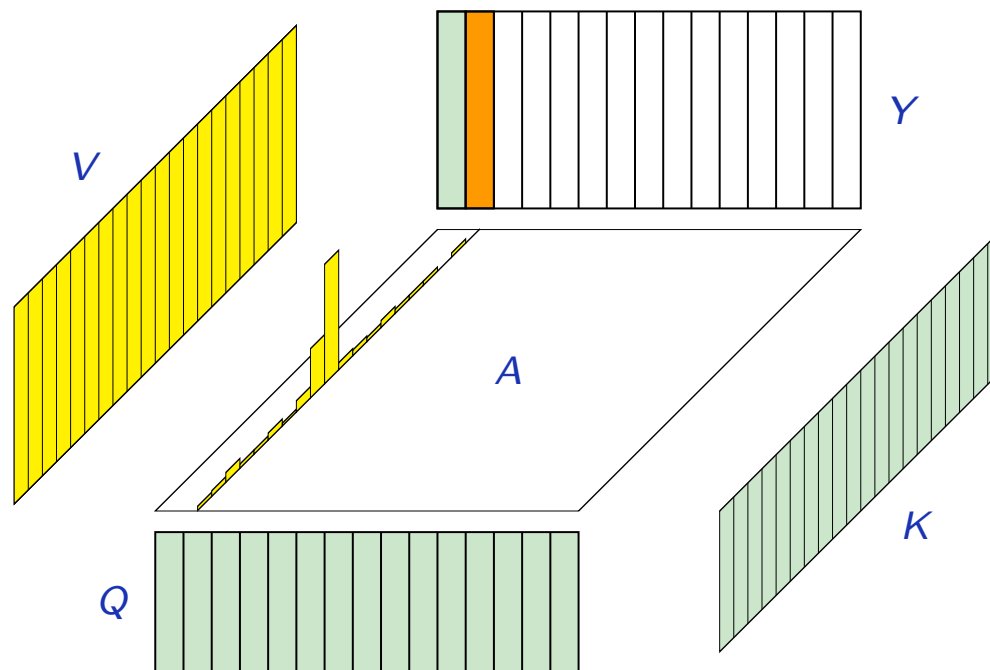
Scaled dot-product attention

$$A_i = \text{softmax} \left(\frac{KQ_i}{\sqrt{D}} \right)$$



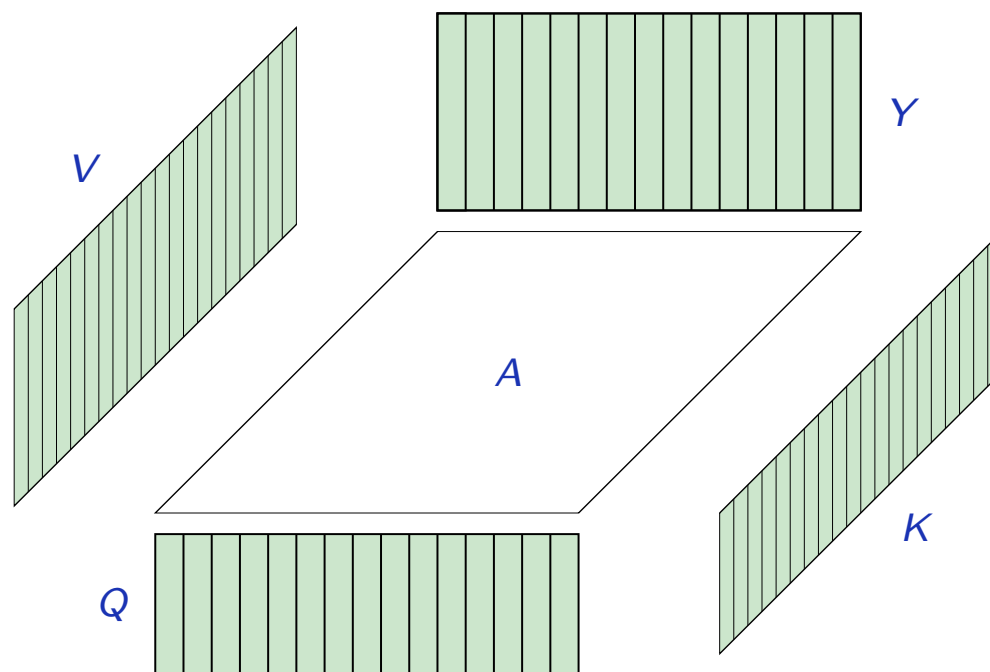
Scaled dot-product attention

$$Y_i = V^T A_i$$



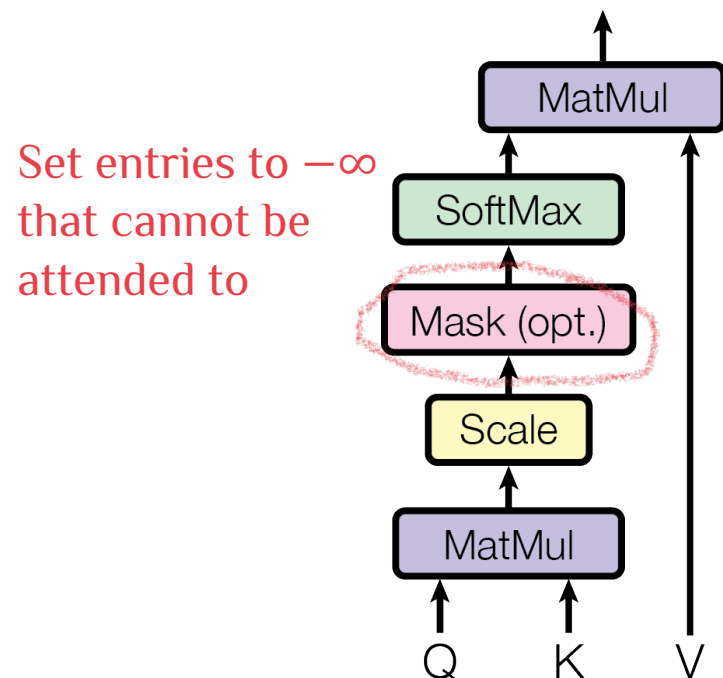
Scaled dot-product attention

$$A_i = \text{softmax} \left(\frac{KQ_i}{\sqrt{D}} \right) \quad Y_i = V^T A_i$$



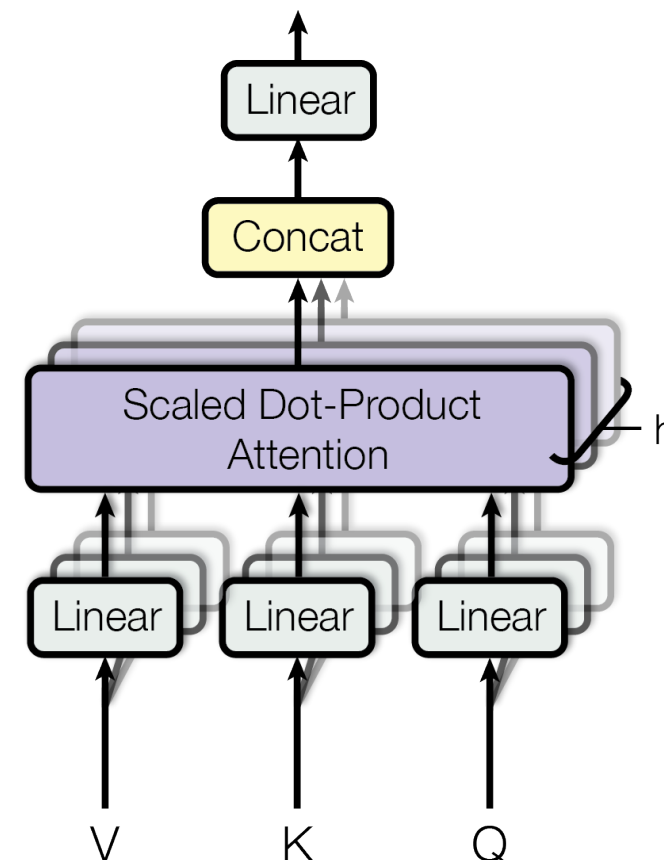
Multi-head attention

Scaled Dot-Product Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-Head Attention



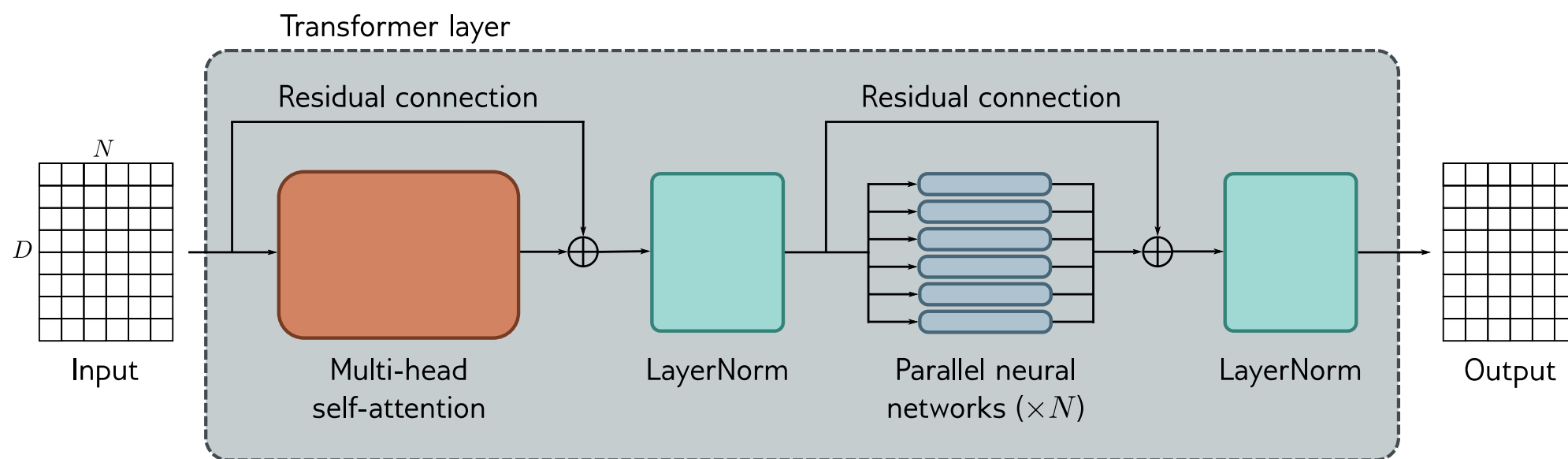
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

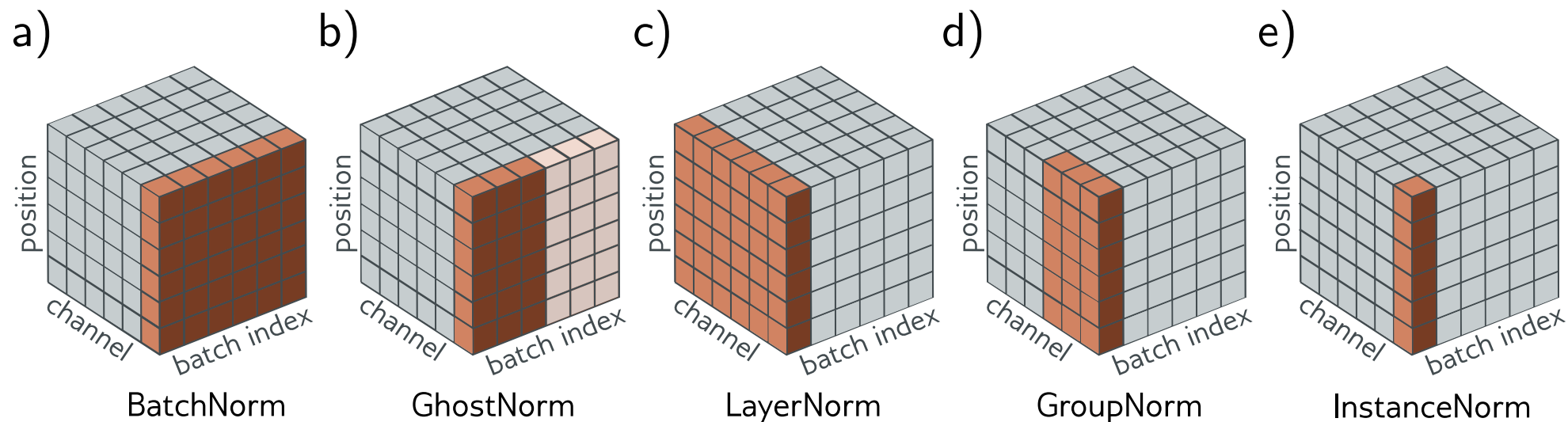
Self attention

In **self-attention** $Q = K = V$

- Connects all positions directly → learn **long-range dependencies**
- **High parallelism**: no sequential dependency like RNNs
- **Interpretability**



Layer normalization



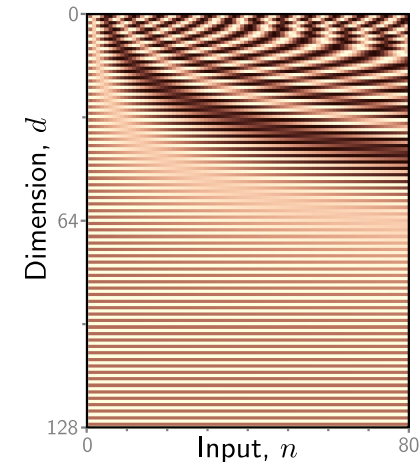
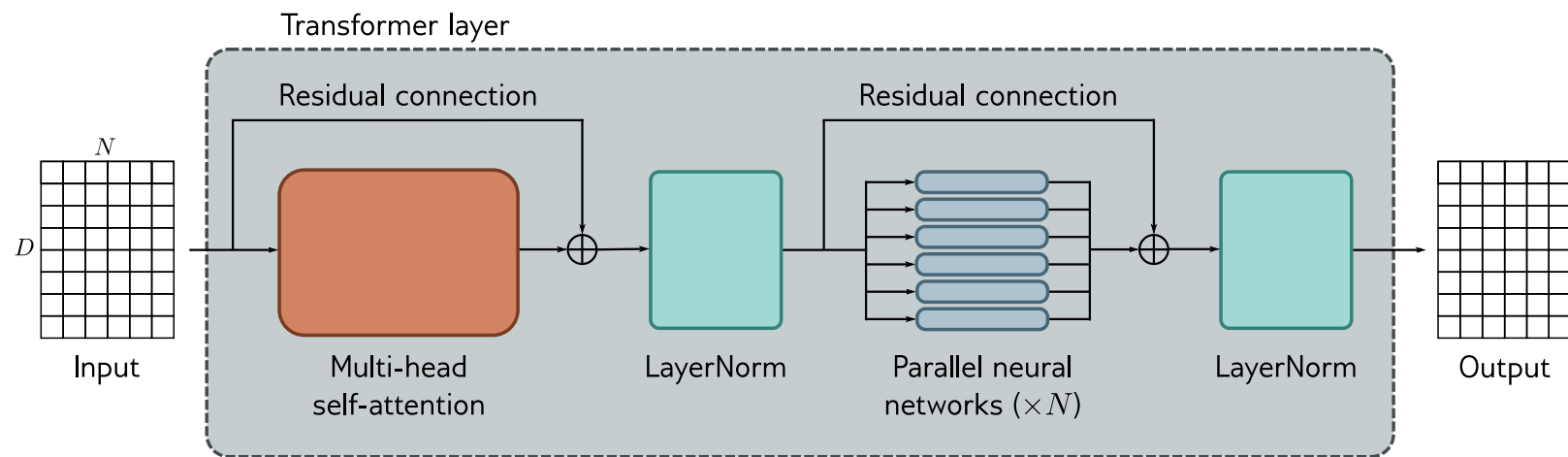
- **BatchNorm:** per-channel normalization based on batch statistics
- **LayerNorm:** per-sample normalization based on channel and pos. stats.

Positional encoding

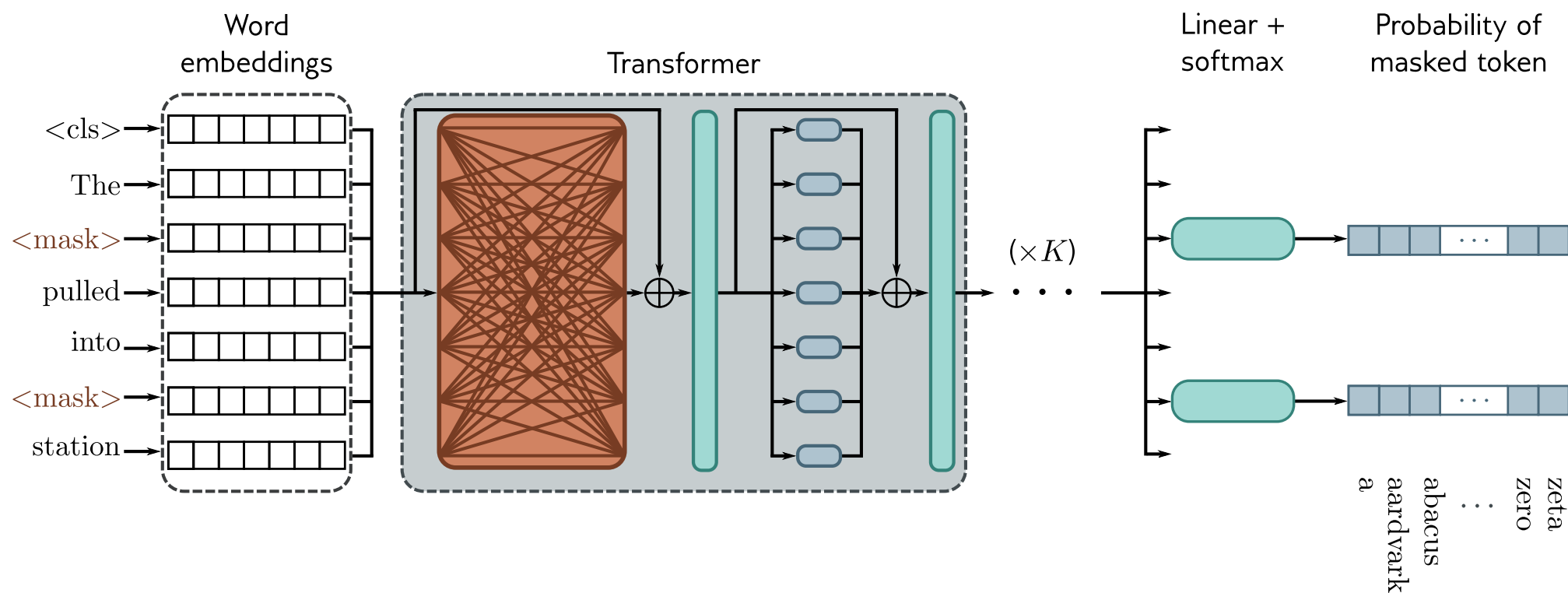
A transform layer **equivariant** with respect to input **permutations**

- (Self-)attention is **equivariant**
- *The remaining sublayers do not connect input positions*

Solution: Add positional encoding to restore order information.

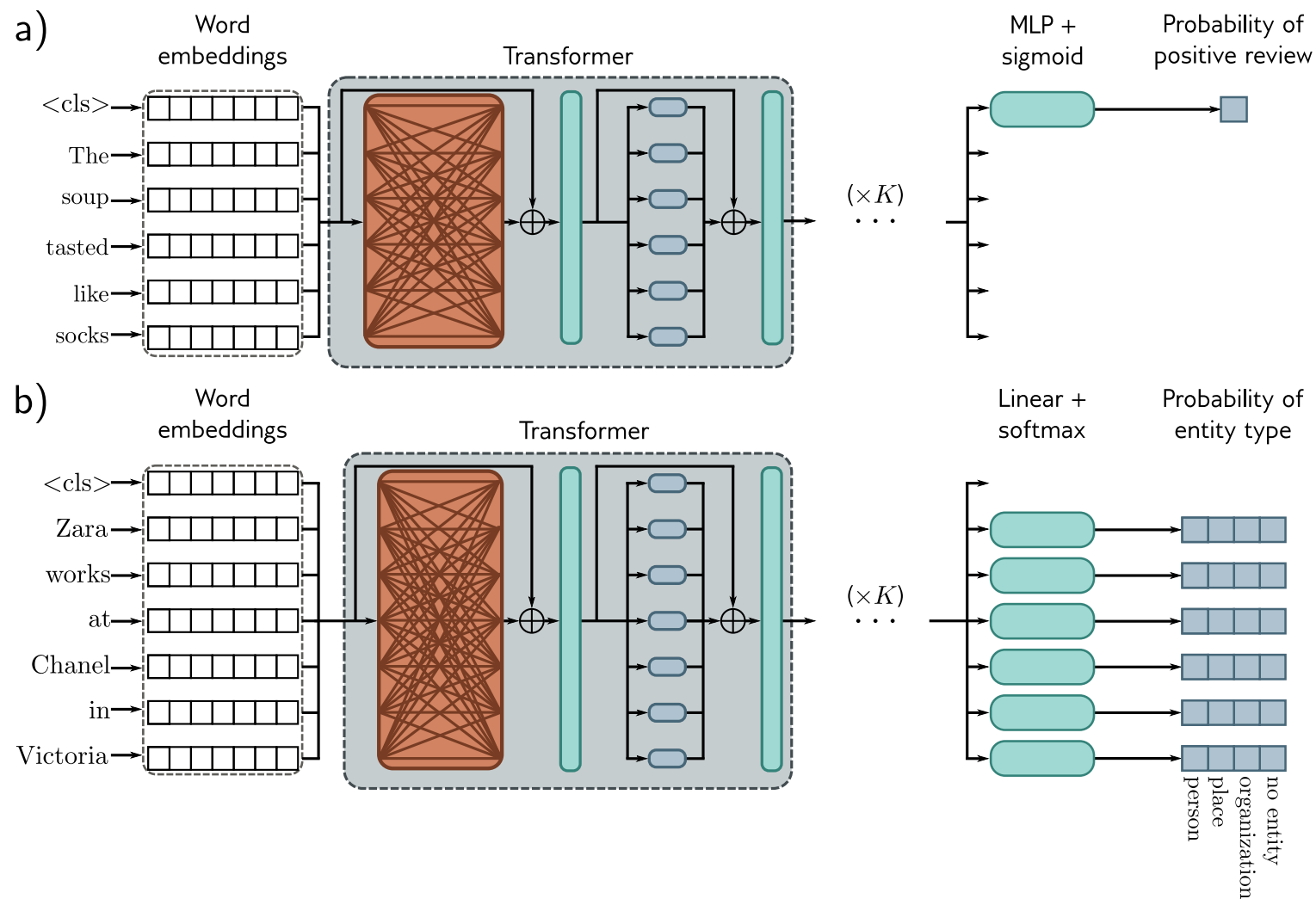


Encoder-only: pre-training

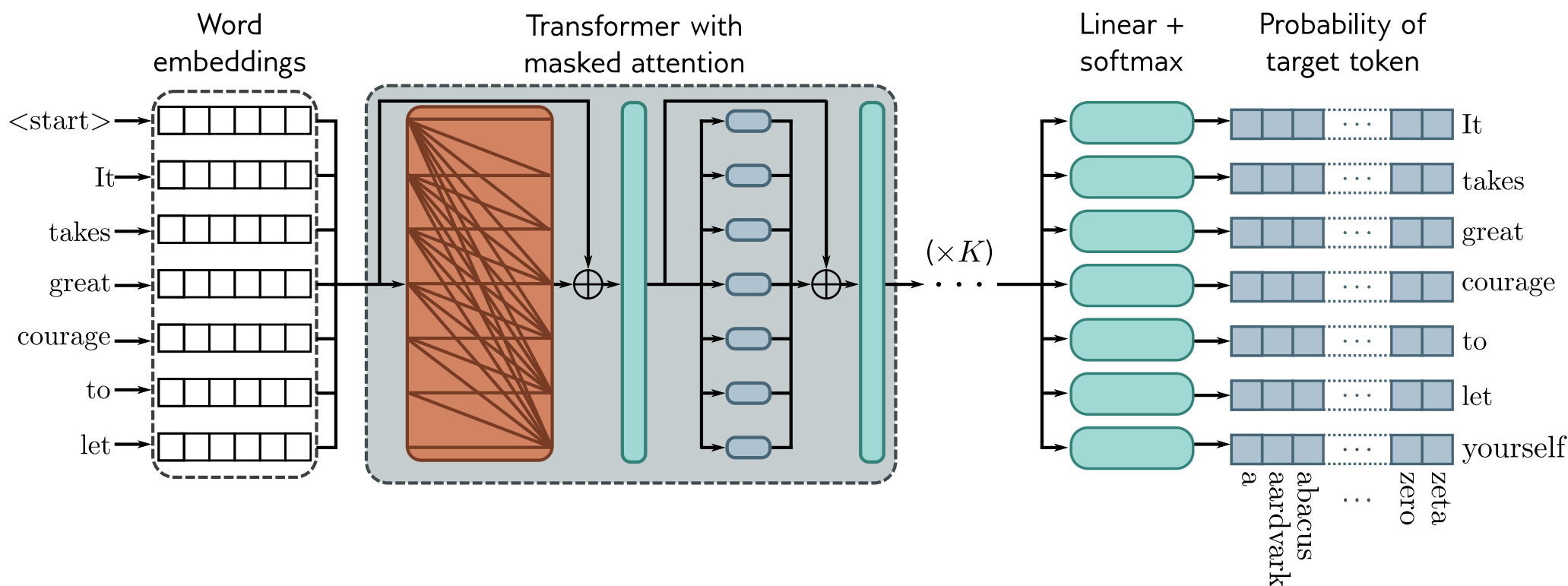


Trained with self-supervision: randomly remove tokens and predict them

Encoder-only: fine-tuning



Decoder-only architecture



Trained with **causal (masked) self-attention** for **next-token** prediction

Encoder-decoder architecture

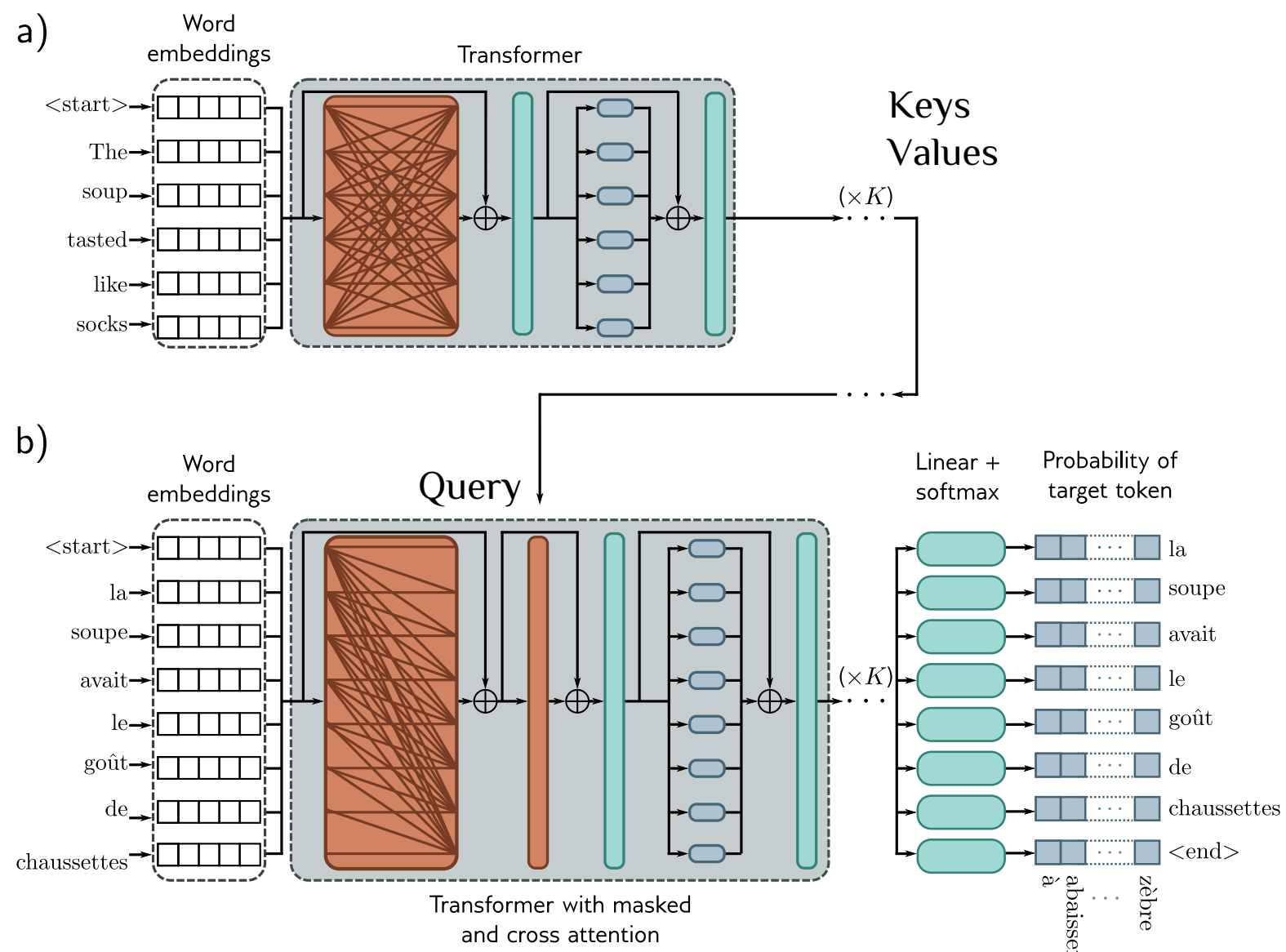


Figure from: Prince SJD. Understanding deep learning. MIT press, 2023.

Today's exercises!

- Three Python notebooks ([Notebook 6.1](#))
 - Language modelling
 - RNNs
 - Attention and Transformers
- Two **problems** from the book on covered material

Thank you!